This problem set is not going to be collected, although solutions will be posted before the quiz. In addition to solving these problems, I recommend writing up solutions as if you were going to submit them. Unlike a problem set collected for credit, though, you are free to discuss with classmates and course staff and get help.

I recommend you first attempt the problems on your own, then discuss with classmates. After that, you may want to consult course staff at help hours or on Ed Discussion. After solutions are posted, consult the solutions and compare to what you wrote, and then discuss the differences with your study group.

All question on this problem set, and also one question on your next topic exam, will require a dynamic programming solution. For any such problem, you **must** do the following; missing one of these can cost you significant credit.

- Give a clear and precise English definition that describes the function you are implementing. Not *how* it works (yet), but rather *what* it solves.
- Give that function a meaningful variable name. This is not [just] me being pedantic; I have found it helps students with this topic if they do this. "OPT" is not a meaningful variable name, nor is "dp" nor "table." Single letters are not meaningful variable names.
- Give a clear recursive formula or algorithm in terms of smaller instances of *exactly* the same problem. If you aren't solving exactly the same problem, you might need to go back to the previous step.
- Describe the iterative running time correctly. This can either be by writing the iterative algorithm (in which case, you can point out where the previous part is within the solution), or by taking your recursive solution, counting the cases, describing the order in which the table would be filled in, and analyzing the time accordingly.

1. College students get a lot of free food at various events. Suppose you have a schedule of the next n days marked with those days when you get a free dinner, and those days on which you must acquire dinner on your own. On any given day you can buy dinner at the cafeteria for \$6. Alternatively, you can purchase one week's groceries for \$20, which will provide dinner for each day that week. However, because you don't have a fridge, the groceries will go bad after seven days (including the day of purchase) and any leftovers must be discarded. Due to your very busy schedule, these are your only three options for dinner each night.

   Give an efficient algorithm to determine, given the schedule of free meals, the minimum amount of money you must spend to make sure you have dinner each night.

   Hint: Start by writing a recursive procedure to determine how you should eat dinner on the last night.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

2. *Skittles* are small candies; each piece of candy is one of five flavors (grape, lemon, orange, strawberry, or green). An exciting new party game, *Skittle Trader*, is sweeping the nation, and the skills you are learning in CompSci 161 will help you to achieve a high score!

   Here are the rules of the game. You are at a party with n of your friends. Each of you has one piece of *Skittles* candy. Your friends form a line. You walk down the line, starting at friend 1 and finishing at friend n. At each friend, you have a choice: do you want to trade the candy you are holding for the one they are holding?

   - If you do not trade candy, your score remains the same.
   - If you and the friend have the same flavor candy, you earn one point.
   - If you and the friend trade, but you had different flavor types, you lose one point. Your score *can* go negative.

   You begin the game with zero points. Your goal is to finish the game with the most possible points.

   For purposes of the input to this problem, the Skittles flavors are numbers $0, 1, 2, 3, 4$. Your input to this problem is an array $C[1 \ldots n]$, where each $C[i] \in [0, 4]$ is the type of candy the ith friend in line is holding. You also have a parameter for the type of candy you begin the game with.

   If you decide to actually play this game with friends, perhaps to test out your algorithm, please use individually wrapped candies instead, such as Starburst. This is nearly the most disgusting question I have given in a class, second only to the Chicken McNuggets question from ICS 46.

3. You've decided to play some music late night at your local club and want to impress. To prepare your strategy, you have reviewed your notes and used statistical analysis to figure out which moves get the most applause ending at certain musical measures. In your findings you've found that at each measure, you can perform one of three different moves.

   - **Jazzy riff:** *This is short and improvised.*
     How long it lasts is constant, but how you play depends on the ending measure.
     Played for 1 measure until the end of measure t, it awards you j_t applause. Note that j is a vector: the measure on which you end the riff affects the applause earned.
   - **Instrumental chorus:** *Your bread and butter.*
     Both how long it lasts and how you play is constant.
     Played for 2 measures until the end of measure t, it awards you C applause. Note that C is a constant: an instrumental chorus always earned you C applause.
   - **Solo:** *You play your heart out!*
     Both how long it lasts and how you play depends on the ending measure.
     Played for k_t measures until the end of measure t, it awards you s_t applause. Note that k and s are vectors: the measure on which you end a solo affects the applause earned.

   Given these statistical models (the vectors j, k, s and the constant C) are correct, you want to figure out **the maximum amount of applause** you can obtain in n musical measures. *Solutions that omit the inclusion of the "solo" option but are otherwise correct will receive reasonable partial credit.*

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

4. When their respective sport is not in season, UCI's student-athletes are very involved in their community, helping people and spreading goodwill for the school. Unfortunately, NCAA[1] regulations limit each student-athlete to at most one community service project per semester, so the athletic department is not always able to help every deserving charity. For the upcoming quarter, we have S student-athletes who want to volunteer their time, and B buses to help get them between campus and the location of their volunteering[2]. There are F projects under consideration; project i requires s_i student-athletes and b_i buses to accomplish, and will generate $g_i > 0$ units of goodwill for the university. Determine which subset of the projects we should do, given the resources we have, to generate the maximum total goodwill.

5. Suppose you are helping to expand the popular chain of Algorithmic Pizza Parlor stores, who are now going to try to gain new customers by advertising. We have a budget of B dollars to spend on advertising, and n possible ways to spend it (billboards, television spots, radio ads, etc). For each possible spending method i, we can allocate anywhere between 0 and B dollars, but must do so in even increments of one dollar (we cannot allocate partial dollars). Our marketing department has come up with a series of functions $\{f_i\}$ to determine how effective (in terms of customers gained) each advertising method will be. If we spend d dollars on advertising method i, we will gain $f_i(d)$ customers. These functions are all non-decreasing: if $d < d'$, then $f_i(d) \leq f_i(d')$. You may assume that $f_i(0) = 0$ for all i, if you would like.

   Given these functions, use **dynamic programming** to design an algorithm that will determine the maximum number of new customers we can gain within our budget.

   Hint: The correct solution is not to figure out where to put the last marginal dollar.

6. Printing Neatly Consider the problem of neatly printing a paragraph with a monospaced font (all characters having the same width) on a printer. The input text is a sequence of n words of lengths $l_1, l_2, \ldots l_n$, measured in characters. We want to print this paragraph neatly on a number of lines that hold a maximum of M characters each. Our criteria of "neatness" is as follows. If a given line contains words i through j, where $i \leq j$, and we leave exactly one space between words, the number of extra space characters at the end of the line is $M - j + i - \sum_{k=i}^{j} l_k$, which must be nonnegative so the words fit on a line. We wish to minimize the sum, over all lines **except** the last, of the cubes of the numbers of extra space characters at the ends of lines. Give a dynamic-programming algorithm to print a paragraph of n words neatly on a printer.

7. We are given an array $A[1 \ldots n]$ for some odd n. Each odd-indexed location contains a positive number, and every even-indexed location contains either a $\times$ or a $+$. This forms an expression of n positive numbers, separated by multiplies and adds, but with the parenthesis removed so it is not clear what the order of operations was intended to be. Our goal is to determine the maximum possible result attainable by adding parenthesis. For example if the expression is: $2 * 15 + 4 * 0.5$ then we would output 34, which is attainable by $2 * (15 + (4 * 0.5))$ (although we only need to return 34, not the expression).

[1] Regulations mentioned in this problem are not necessarily accurate to reality and should be considered parody.

[2] The NCAA also won't permit student-athletes to use their own vehicles for these purposes. Furthermore, each bus can only be used for one project.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

8. Matrix Chain Multiplication Suppose we wish to compute the product of n matrices: $A_1A_2 \dots A_n$. Because matrix multiplication is associative, any parenthsization of this expression will yield the same result. We say a product of matrices is **fully parenthesized** if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses. For example, if $n = 4$, we can fully parenthesize the product in five ways:

   - $(A_1(A_2(A_3A_4)))$
   - $(A_1((A_2A_3)A_4))$
   - $((A_1A_2)(A_3A_4))$
   - $((A_1(A_2A_3))A_4)$
   - $(((A_1A_2)A_3)A_4)$

   While each of these produces the same *resulting product*, they do not require the same amount of work to produce.

   The following is a quick reminder of how we multiply two matrices. First, it must be the case that the number of columns in A is equal to the number of rows in B. If A is $p \times q$ and B is $q \times r$, then the resulting matrix will be $p \times r$ and will require pqr scalar multiplications to produce.

   Example: Suppose we wish to multiply three matrices $A_1A_2A_3$. The dimensions, respectively, are $10 \times 100, 100 \times 5, 5 \times 50$. Would it be better to multiply this as $((A_1A_2), A_3)$ or $(A_1(A_2A_3))$? That is, which of those options will require fewer *scalar* multiplications?

   For this problem, we are looking into deciding which order to multiply the matrices; the actual multiplications of them is not being done here.

   Input: If we are to multiply n matrices, we need only the relative dimensions. We know that A_i's columns must equal the number of rows in A_{i+1} for our input to be valid, so we need only to know the number of rows in each matrix, plus the number of columns in A_n. The rows in each matrix are equal to the number of columns in the previous matrix.

   Thus, our input is just a list of $n + 1$ positive integers: $d_0d_1 \dots d_n$, where matrix i is of dimension $d_{i-1} \times d_i$ for i in the range $1 \dots n$.

9. In the **Euclidean** traveling salesperson problem,m we are given n points in the plane and we wish to find the shortest closed tour that connects all n points. Suppose we simplify the problem by restricting ourselves to **bitonic** tours: that is, we start at the leftmost point, go strictly rightward to the rightmost point (visiting a subset of the other vertices in between), then go strictly leftward (visiting any unvisited points along the way). Give an algorithm to find the optimal bitonic tour.

   You may assume the points are sorted by x coordinate, and that all x coordinates are distinct.

   Hint: At any given point, you want to maintain the optimal possibilities for the two parts of the tour.

   Note: I see an amazing number of wrong answers to this online. Please be wary of getting incorrect information.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

10. Suppose we are given an n-digit positive integer X. We repeatedly remove one digit from either end of X (your choice). The *square depth* of X is the maximum number of perfect squares that you can see during this process.

    For example, the number 32492 has square-depth 3, by the following sequence of removals: remove the 2 at the end, leaving us with 3249 (which is 57^2). Then remove the 9 from the end, leaving us with 324 (which is 18^2). Then remove the 3 from the front, leaving us with 24, which is not a perfect square. Then remove the 2 from the front, leaving us with 4 (which is 2^2). The last removal leaves us with an empty number, and we are done.

    Use dynamic programming to design an efficient algorithm that takes as input an n-digit positive integer X, represented as a vector $X[1 \dots n]$ of n decimal digits, and returns its square depth. It is enough to return that as a number.

    You may assume you have access to a subroutine IsSquare(i, j) that returns 1 if $X[i \dots j]$ is a perfect square, and zero otherwise. This function runs in time $\mathcal{O}(1)$.

11. When classes were much smaller, I had a candy jar in my office. This question came from that era. Students would come to office hours not only seeking knowledge, but also because I give them candy; each student gets one piece of candy for each office hours attended. I have a candy jar in my office that can hold C pieces of candy. At any point, I can ask the department to add candy to the jar; they'll charge me a fixed price $\$P$ for doing so, regardless of how many pieces of candy I ask them for. They also don't like letting candy stay in the jar, getting stale; to discourage me from letting that happen, they charge me $\$d$ for each leftover piece after every office hour.

    For the next quarter, I have n office hours planned; at the ith office hour, I project that s_i students will show up, each of whom will get one piece of candy. For this problem, assume that the projected attendance at office hours is accurate. At the start of the quarter, my candy jar is empty, and I aim to have it empty at the end of the semester.

    While it is possible to solve this problem in time polynomial only in n, if I were collecting this for credit, full credit would be given for a correct solution with running time $\mathcal{O}(nC)$ (assuming the other required parts of the problem are fulfilled)

Recommended Book Problems

If you want additional problems to try, consider trying R-12.7, R-12.8, C-12.1, C-12.9, C-12.16, A-12.1, A-12.2, A-12.3, A-12.4, A-12.6, A-12.10 in the textbook of Goodrich and Tamassia or Chapter 3, problems 1, 2, 3, 6, 9, 11, 12, 13, 16, 17, 18, 23, 32, 33, 35, 46, or 49 in the textbook of Erickson. For a big challenge, try question 28 in that chapter.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.